

36 — DAIL Experiment Specification

Version 1.0

DAIL Subsystem — Experimentation & Evaluation Specification

Experiment rule: an experiment may measure and learn from alternative behavior; it must never silently convert experimental results into verified, approved, promoted, or trusted infrastructure state.

Document	Value
System	CloudShield AI
Specification	36 — DAIL Experiment
Version	1.0
Scope	Controlled experimentation for DAIL dependency, impact, verification, evidence, and promotion-related behavior
Subsystem	DAIL within CloudShield AI
Status	Engineering Baseline
Depends On	00–35

Document Map

- 1. Purpose and Scope
- 2. Experimentation Objectives and Principles
- 3. DAIL Experiment Boundary
- 4. Experiment Lifecycle
- 5. Experiment Types
- 6. Hypothesis Model
- 7. Experiment Definition Schema
- 8. Variables and Treatment Model
- 9. Control and Baseline Design
- 10. Experiment Eligibility
- 11. Safety and Governance Gates
- 12. Environment and Isolation
- 13. Test Data and Fixtures
- 14. Dependency Graph Experiments
- 15. Impact Analysis Experiments
- 16. Verification Experiments
- 17. Promotion and Trusted-State Experiments
- 18. Evidence and Audit Experiments
- 19. Lifecycle and Invariant Experiments
- 20. DAIL Performance Experiments
- 21. Reliability and Failure Experiments
- 22. Concurrency and Consistency Experiments
- 23. AI/LLM-Assisted DAIL Experiments
- 24. Experiment Harness Architecture
- 25. Experiment Execution
- 26. Metrics and Measurement

- 27. Statistical and Analytical Rules
- 28. Experiment Results
- 29. Experiment Reproducibility
- 30. Experiment Failure and Abort
- 31. Experiment Rollback and Cleanup
- 32. Experiment-to-Production Boundary
- 33. Experiment Security
- 34. Observability
- 35. Experiment Evidence and Reporting
- 36. CI/CD Integration
- 37. MVP Experiment Baseline
- 38. Implementation Sequence
- 39. Acceptance Criteria
- 40. Traceability
- 41. Engineering Decisions
- 42. Definition of Done
- 43. Glossary

1. Purpose and Scope

This specification defines how CloudShield AI experiments are designed, executed, measured, evaluated, and closed for the DAIL subsystem. DAIL experimentation improves dependency modeling, impact analysis, verification quality, evidence quality, lifecycle safety, performance, and reliability without weakening the product's production safety boundary.

Experimentation is a controlled engineering capability rather than an unrestricted research environment. Algorithms, heuristics, graph strategies, verification policies, evidence strategies, and AI-assisted techniques may be compared, but production authority remains outside the experiment harness.

1.1 In Scope

- DAIL dependency and graph experiments.
- Impact/blast-radius algorithm experiments.
- Verification strategy, ordering, confidence, and failure-mode experiments.
- Lifecycle/invariant experiments.
- Evidence quality and completeness experiments.
- Performance, concurrency, resilience, and reconciliation experiments.
- AI/LLM-assisted DAIL experiments where AI remains non-authoritative.
- Experiment metadata, control/treatment configuration, metrics, statistical analysis, evidence, and reproducibility.
- Criteria for moving an experimentally validated implementation into normal engineering releases.

1.2 Out of Scope

- Unrestricted production experimentation on customer infrastructure.
- Using experiments to bypass DAIL verification or governance.
- Automatically promoting an experimental algorithm solely because it improves a metric.
- Training a foundation model as a prerequisite for DAIL experimentation.

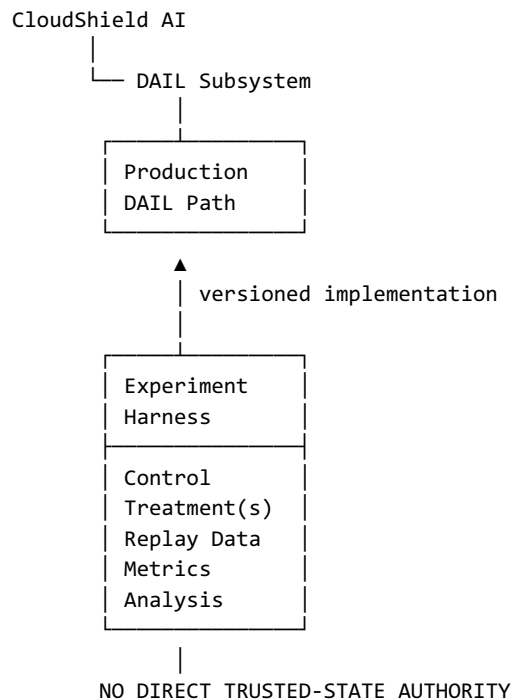
2. Experimentation Objectives and Principles

Objective	Requirement	Expected Outcome
-----------	-------------	------------------

Scientific validity	Define hypothesis, baseline, treatment, metrics, and success criteria before execution.	Results are interpretable.
Safety	Keep experiments isolated from production authority.	Experimental failure cannot create trusted state.
Reproducibility	Version code, data, configuration, graph, policy, and environment.	Results can be recreated.
Traceability	Link experiment to source, release, DAIL state, and evidence.	Decision history is reconstructable.
Comparability	Use consistent baselines and controlled variables.	Treatment effects are attributable.
Risk awareness	Measure false positives, false negatives, uncertainty, and unsafe outcomes.	Optimization does not hide safety regressions.
Operational value	Prefer experiments tied to measurable product outcomes.	Research translates into engineering decisions.

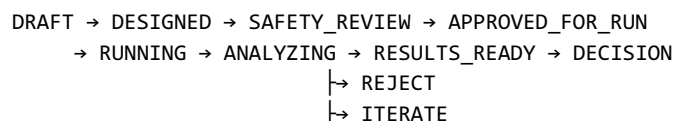
A better average metric is not sufficient if safety-critical tail behavior becomes worse.

3. DAIL Experiment Boundary



- Experiments may invoke DAIL logic in sandbox, replay, or explicitly controlled test-cloud scopes.
- Experimental treatment code cannot directly authorize production promotion.
- Experimental outputs are labeled experimental and cannot be consumed as authoritative verification without passing the normal production path.
- Any real-cloud experiment uses dedicated scopes and explicit safety limits.

4. Experiment Lifecycle



↳ CANDIDATE_FOR_ENGINEERING
→ NORMAL TESTING → RELEASE → PRODUCTION

- Experiments may be stopped by safety controls at any point.
- Results do not automatically become implementation changes.
- Engineering adoption requires normal code review, system testing, security review, deployment, and lifecycle controls.
- Experiment status is durable and auditable.

5. Experiment Types

Type	Purpose	Example
Algorithm	Compare DAIL computation strategies.	Graph traversal vs indexed impact query.
Heuristic	Compare deterministic heuristics.	Dependency-confidence thresholds.
Verification	Evaluate verification strategies.	Targeted checks vs full sequence.
Reliability	Measure failure behavior.	Provider timeout during impact analysis.
Performance	Measure latency/resources.	Large dependency graph traversal.
Concurrency	Test simultaneous state changes.	Parallel verification requests.
Data Quality	Evaluate dependency/observation quality.	Conflicting provider relationships.
Evidence	Compare evidence strategies.	Evidence schema/collection strategy.
AI-Assisted	Evaluate bounded AI support.	AI-assisted dependency explanation.
UX/Operator	Measure analyst effectiveness.	Impact visualization alternatives.

6. Hypothesis Model

Field	Requirement
Problem	State the engineering problem and why it matters.
Hypothesis	Predict measurable treatment effect.
Baseline	Define current behavior/control.
Treatment	Define exactly what changes.
Population	Define assets, candidates, graphs, or runs.
Primary Metric	One principal metric for the main decision.
Secondary Metrics	Supporting quality, safety, performance, and cost measures.
Safety Metrics	Metrics that must not regress.
Success Threshold	Predefined decision threshold.
Stop Criteria	Conditions terminating the experiment.
Decision	Adopt, reject, iterate, or investigate.

Example: treatment reduces median impact-analysis latency by $\geq 25\%$ while maintaining 100% agreement on critical dependency cases and introducing zero unsafe bounded-impact classifications.

7. Experiment Definition Schema

Field	Description
experiment_id	Stable unique identifier.
name	Human-readable name.
owner	Responsible engineering owner.
objective	Problem being investigated.
hypothesis	Testable prediction.
baseline_version	Exact control implementation.
treatment_versions	Exact experimental implementations.
dataset_version	Versioned input population/fixtures.
graph_version	Dependency graph snapshot/version.
policy_version	Relevant DAIL/security policy.
environment	Sandbox/test/staging or explicitly approved scope.

metrics	Primary, secondary, and safety metrics.
sample_plan	Population and sampling rules.
run_config	Execution parameters.
safety_limits	Resource/action bounds.
success_criteria	Predeclared thresholds.
stop_criteria	Abort conditions.
result	Final outcome and evidence reference.

8. Variables and Treatment Model

- Independent variables are explicit and versioned.
- Dependent variables are measurable outcomes.
- Record confounders such as graph size, provider, region, asset class, policy version, and dependency freshness.
- Treatments should change as few variables as practical.
- Use randomization where appropriate; use deterministic stratification for important security classes.
- Evaluate safety-critical classes separately from aggregate averages.

Variable Class	Examples
Independent	Traversal algorithm, confidence threshold, verification ordering.
Dependent	Latency, recall, precision, false-negative rate, evidence completeness.
Confounder	Graph size, resource type, region, dependency freshness.
Safety Constraint	Unsafe classification count, unauthorized transition count.
Cost Constraint	CPU, memory, cloud API calls, LLM tokens.
Operational	Queue depth, retries, worker utilization.

9. Control and Baseline Design

- Baseline represents the current accepted implementation or documented reference.
- Control and treatment receive equivalent inputs.
- Keep ordering, graph snapshots, policy versions, and relevant configuration constant unless intentionally varied.
- Caches are disabled or applied consistently.
- Stochastic components use recorded seeds or repeated runs.
- Production behavior is not an uncontrolled comparison when data/configuration differs materially.

10. Experiment Eligibility

- Experiment has an owner and defined objective.
- Safety and authorization reviews complete before execution.
- Input data is approved.
- Production mutation is prohibited unless separately authorized.
- Resource, time, cost, and API-call limits are defined.
- Stop criteria are executable where practical.
- Rollback/cleanup is defined before execution.
- Outputs are labeled experimental and cannot be mistaken for authoritative results.

11. Safety and Governance Gates

Gate	Required Check
Scope	Authorized tenant/test scope only.

Identity	Least-privileged experiment identity.
Mutation	No uncontrolled production mutation.
Data	Dataset and sensitive-data handling approved.
AI	Applicable AI guardrails/evaluation.
DAIL	Treatment cannot bypass lifecycle invariants.
Cost	Budget/API/resource ceiling.
Abort	Automatic/manual stop available.
Evidence	Actions/results recorded.
Adoption	Production adoption uses normal engineering gates.

Experiment approval is permission to run an experiment, not approval of its results.

12. Environment and Isolation

Environment	Use
Local	Algorithm/unit experiments with synthetic fixtures.
Test	Automated multi-component experiments.
Sandbox AWS	Controlled provider behavior and real API characteristics.
Staging	Production-like evaluation of release candidates.
Production	No experimental mutation by default; only explicitly approved safe observation/probe experiments.

- Experimental resources use dedicated identifiers/tags.
- Experiment tenants/scopes are isolated from ordinary development data.
- Artifacts and datasets are versioned separately from production state.
- Production secrets are not copied into experiment environments.

13. Test Data and Fixtures

- Canonical DAIL fixtures cover simple, chained, fan-out, fan-in, cyclic, missing, stale, and conflicting dependencies.
- Include small and large graph datasets.
- Include candidate versions with known impact ground truth.
- Include pass, fail, inconclusive, timeout, and authorization-denied verification outcomes.
- Include provider failures and eventual consistency.
- Maintain adversarial datasets for identity confusion, dependency poisoning, stale observations, and malicious metadata.
- Ground-truth datasets are immutable once used in a published comparison; create a new version for changes.

14. Dependency Graph Experiments

- Compare graph construction algorithms on identical observations.
- Measure edge precision, recall, confidence calibration, freshness, construction time, and storage cost.
- Test incomplete provider relationships.
- Test conflict resolution and ambiguous identity.
- Ensure derived edges do not appear authoritative.
- Measure transitive traversal correctness on ground-truth graphs.

Metric	Safety Interpretation
Edge Precision	False edges can create unnecessary blast radius.
Edge Recall	Missing critical edges can create unsafe bounded-impact results.
Confidence Calibration	Low-confidence edges must remain identifiable.

Freshness	Stale edges must remain identifiable.
Conflict Rate	Conflicts should surface rather than silently merge.
Traversal Correctness	Critical reachable dependencies must not be missed.

15. Impact Analysis Experiments

- Evaluate direct vs transitive impact accuracy.
- Measure candidate-specific impact computation.
- Test unknown/partial dependency data.
- Compare conservative vs narrow strategies.
- Measure false-bounded-impact rate as a safety metric.
- Test invalidation when dependencies/candidate versions change.
- Bind impact results to exact graph/context versions.

Candidate V + Graph G + Policy $P \rightarrow$ Impact $I(V,G,P)$. Material changes to $V/G/P$ invalidate I .

16. Verification Experiments

- Compare verification sequences, ordering, and targeted-check strategies.
- Measure detection of intentionally introduced faults.
- Measure false-pass and false-fail rates.
- Test partial verification and stale observations.
- Test repeated verification for determinism/idempotency.
- Measure time-to-verification and external API consumption.

Verification success is never defined merely as agreement with the treatment; ground truth and explicit safety properties remain authoritative.

17. Promotion and Trusted-State Experiments

- Run promotion experiments in sandbox/staging by default.
- Test invalidation from candidate changes, stale approvals, changed dependencies, or changed observations.
- Measure false-promotion prevention.
- Test ambiguous cloud mutation and reconciliation.
- Prove experimental results cannot directly write trusted state.
- Evaluate governance usability separately from authority enforcement.
- Any proposed promotion algorithm must pass normal DAIL verification/governance before production adoption.

18. Evidence and Audit Experiments

- Compare evidence strategies for completeness, retrieval cost, and analyst usefulness.
- Measure actor, scope, version, timestamp, and correlation identifiers.
- Test evidence integrity/immutability.
- Test reconstruction of an experiment from stored evidence.
- Treat missing critical evidence as a safety failure.

19. Lifecycle and Invariant Experiments

- Use state-machine exploration for valid/invalid DAIL transitions.
- Generate concurrent candidate, verification, approval, and promotion sequences.
- Check invariants after every operation.
- Test interrupted transactions and worker crashes.
- Use property-based generation where practical.
- Any invariant violation aborts the experiment and creates a defect.

Examples:

```
verified_version == candidate_version at verification time
approval invalidates after material protected-input change
promotion requires valid verification
trusted state requires authoritative confirmation
experimental execution cannot bypass authorization
```

20. DAIL Performance Experiments

Workload	Measures
Small graph	Baseline latency/memory.
Medium graph	Throughput/tail latency.
Large graph	Scalability/resource saturation.
High concurrency	Queueing/contention/correctness.
Provider throttling	Retry amplification/completion time.
Evidence-heavy run	Storage/serialization overhead.
AI-assisted run	LLM latency/cost vs deterministic baseline.

- Report p50/p95/p99 where meaningful.
- Separate cold-start and warm-cache behavior.
- Record worker count, database configuration, infrastructure size, and cloud limits.
- Do not optimize average latency by accepting unsafe tail behavior.

21. Reliability and Failure Experiments

- Inject provider timeout, throttling, permission denial, malformed/partial response, queue failure, database failure, object-storage failure, and AI outage.
- Measure completion, retries, reconciliation, and correctness after recovery.
- Verify no failure path silently marks verification/promotion successful.
- Test safe degradation and explicit unknown/inconclusive states.
- Test restart/resume of interrupted runs.

22. Concurrency and Consistency Experiments

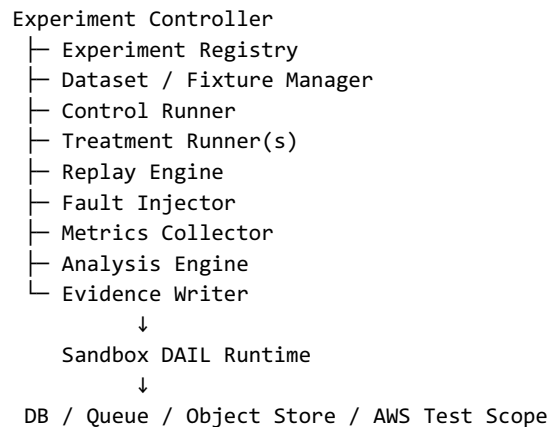
- Run concurrent discovery and verification.
- Run candidate updates against concurrent verification.
- Run competing approval/promotion attempts.
- Test duplicate messages and at-least-once delivery.
- Validate optimistic concurrency/version checks.
- Test stale workers.
- Compare final state with serialized reference behavior where applicable.

23. AI/LLM-Assisted DAIL Experiments

- Evaluate AI for dependency explanation, evidence summarization, anomaly explanation, candidate reasoning, or prioritization support.
- AI outputs are structured, validated, and labeled as recommendations/explanations.
- AI is not authoritative dependency truth without deterministic corroboration.
- Measure hallucination, grounding, structured validity, safety, latency, cost, and consistency.
- Test prompt injection from resource metadata, tickets, code, documents, and graph labels.
- Compare AI-assisted behavior against deterministic baselines.
- Model/provider changes require experiment/version metadata and safety evaluation.

AI-assisted experiments evaluate assistance quality; they do not delegate verification or promotion authority to the model.

24. Experiment Harness Architecture



- Record exact implementation, dataset, configuration, and environment versions.
- Control and treatment share a common input contract.
- Fault injection is deterministic where practical.
- Analysis is separated from execution for independent review.

25. Experiment Execution

1. Create experiment definition and version all inputs.
2. Validate eligibility and safety gates.
3. Provision isolated environment/resources.
4. Load immutable dataset/graph/policy versions.
5. Run baseline/control.
6. Run treatment(s).
7. Collect metrics, logs, traces, events, audit, and evidence.
8. Run post-execution integrity checks.
9. Analyze predefined primary and safety metrics.
10. Record decision and supporting evidence.
11. Clean up temporary resources.
12. Create engineering follow-up if adoption is justified.

26. Metrics and Measurement

Metric Class	Examples
Correctness	Precision, recall, ground-truth agreement, invariant violations.
Safety	False-negative rate, unsafe bounded-impact rate, unauthorized transitions.
Verification	Detection rate, false-pass rate, inconclusive rate.
Performance	p50/p95/p99 latency, throughput, CPU, memory.
Reliability	Failure rate, retry count, recovery success.
Freshness	Observation age, dependency age, invalidation delay.
Evidence	Completeness, integrity, reconstruction success.
Cost	Cloud API calls, storage, compute, LLM tokens/cost.
Operator Value	Time-to-decision, explanation usefulness, review effort.

Safety metrics are constraints, not optional secondary KPIs.

27. Statistical and Analytical Rules

- Define the primary metric and success threshold before running.
- Choose sample size/run count appropriate to variance and decision importance.
- Report uncertainty/confidence intervals where meaningful.
- Do not cherry-pick favorable runs.
- Distinguish independent samples from repeated measurements.
- Segment results by security-critical asset classes and edge cases.
- Aggregates must not hide catastrophic failures in small critical populations.
- When formal significance is inappropriate, use explicit engineering thresholds and document the decision method.

28. Experiment Results

Result	Meaning
ADOPT	Treatment meets success and safety criteria and may enter normal engineering pipeline.
REJECT	Treatment fails criteria or introduces unacceptable risk.
ITERATE	Promising but insufficient evidence; redesign and rerun.
INCONCLUSIVE	Data quality/sample/variance prevents reliable decision.
ABORTED	Safety/resource/infrastructure/integrity stop condition occurred.
SUPERSEDED	Replaced by a newer controlled experiment.

- Every result includes supporting metrics/evidence.
- ADOPT does not directly deploy the treatment.
- Production adoption uses Specification 34 deployment and Specification 35 testing.

29. Experiment Reproducibility

- Record source revision, artifact digest, experiment definition, dataset, graph snapshot, policy, environment, dependencies, random seeds, and runtime configuration.
- Record external provider versions/configuration when material.
- Store enough evidence to reconstruct inputs and outputs.
- Changed data creates a new run identity.
- Published results reference immutable experiment-run records.

30. Experiment Failure and Abort

- Automatic abort conditions include invariant violation, unauthorized action attempt, resource ceiling breach, uncontrolled cloud mutation, critical evidence-write failure, and safety-metric breach.
- Operators can stop experiments immediately.
- Aborted runs remain recorded.
- After abort, inspect partial side effects.
- Cloud state is reconciled before cleanup is considered complete.

Safety Breach → STOP → FREEZE/CONTAIN → OBSERVE ACTUAL STATE → RECONCILE → PRESERVE EVIDENCE → DEFECT/INCIDENT REVIEW

31. Experiment Rollback and Cleanup

- Algorithm-only experiments normally require no cloud rollback.
- Sandbox mutations use explicit cleanup/restore procedures.
- Temporary database records, queues, objects, and infrastructure are tagged and safely removed.
- Cleanup failure is recorded/escalated.
- Never delete evidence to make cleanup appear successful.
- Production adoption is never used as cleanup for an unfinished experiment.

32. Experiment-to-Production Boundary

Experiment Result → Engineering Review → Code/Config Change → Normal CI → System Testing → Security/AI/DAIL Gates → Staging → Release Approval → Production → Runtime Verification

No direct experiment → production path exists.

33. Experiment Security

- Least-privilege experiment identities.
- Separate credentials/secrets from production.
- Tenant/scope isolation.
- Input sanitization and prompt-injection defenses.
- Network restrictions and egress controls.
- Resource quotas and cost limits.
- Immutable experiment evidence.
- Security scanning of experimental code in shared environments.
- Experiment harnesses are not general-purpose cloud mutation tooling.

34. Observability

- Every run has experiment_id, run_id, treatment_id, source_revision, artifact_digest, dataset_version, and correlation IDs.
- Metrics distinguish control and treatment.
- Logs include execution phase and safety events.
- Traces cover harness → DAIL → persistence/cloud/AI boundaries.
- Critical events produce audit/evidence records.
- Dashboards expose health, safety stops, resource use, and result status.

35. Experiment Evidence and Reporting

- Report includes objective, hypothesis, design, baseline, treatment, dataset, environment, metrics, results, safety analysis, limitations, and decision.
- Evidence links results to raw run metadata.
- Reports explicitly identify failed/inconclusive experiments.
- Security-sensitive experiments preserve relevant audit records.
- Published results use immutable run/version references.

Report Section	Required Content
Executive Summary	Question, result, decision.
Design	Control/treatment, variables, sampling.
Results	Primary, secondary, safety metrics.
Analysis	Interpretation, uncertainty, segmentation.
Risk	Failure modes and limitations.
Reproducibility	Versions and run configuration.
Decision	Adopt/reject/iterate/inconclusive/aborted.
Evidence	Immutable artifact/run references.

36. CI/CD Integration

- Experiment code passes normal engineering checks before shared-environment execution.
- Fast experiment smoke tests run in CI for affected DAIL modules.
- Nightly/scheduled suites execute larger datasets and performance/reliability experiments.
- Release candidates may require relevant experiment evidence for algorithmic changes.
- Experiment results cannot bypass Specification 35 system-test gates.
- Adopted treatment becomes ordinary source-controlled code under Specification 34 deployment controls.

37. MVP Experiment Baseline

- Versioned experiment registry and run metadata.
- Deterministic synthetic DAIL datasets and ground-truth dependency graphs.
- Control/treatment replay harness.
- Dependency graph and impact-analysis comparisons.
- Lifecycle/invariant property tests.
- Verification false-pass/false-fail evaluation.
- Concurrency and duplicate-processing experiments.
- AWS provider failure and reconciliation experiments in controlled scope.
- Evidence completeness and reconstruction experiments.
- AI-assisted DAIL evaluation for bounded tasks.
- Performance baseline for small/medium/large graphs.
- Basic fault injection for queue/database/AWS/LLM failures.
- Immutable experiment evidence and reporting.
- Promotion-boundary tests proving experimental output cannot create trusted state.

38. Implementation Sequence

13. Define experiment metadata schema and registry.
14. Build deterministic DAIL fixture and ground-truth dataset library.
15. Implement control/treatment replay harness.

16. Implement dependency graph experiment runners.
17. Implement impact-analysis experiment runners.
18. Implement lifecycle/invariant property harness.
19. Implement verification quality experiments.
20. Implement concurrency and failure-injection harnesses.
21. Implement evidence/audit collection.
22. Implement performance and cost measurement.
23. Add controlled AWS sandbox experiments.
24. Add AI-assisted DAIL evaluation datasets and safety tests.
25. Integrate experiment smoke gates into CI/CD.
26. Create result/report workflow.
27. Establish adoption review process.
28. Run scheduled benchmark and regression experiments.

39. Security and Functional Acceptance Criteria

ID	Acceptance Criterion
DAIL-EXP-AC-001	Every experiment has versioned definition, owner, hypothesis, baseline, treatment, and success criteria.
DAIL-EXP-AC-002	Experimental execution is isolated from production trusted-state authority.
DAIL-EXP-AC-003	Experiment identities are least-privileged and cannot bypass DAIL governance.
DAIL-EXP-AC-004	Control and treatment receive equivalent versioned inputs unless explicitly varied.
DAIL-EXP-AC-005	Safety metrics include unsafe bounded-impact and unauthorized-transition checks where applicable.
DAIL-EXP-AC-006	Critical invariant violations automatically abort the experiment.
DAIL-EXP-AC-007	Ambiguous sandbox cloud mutations enter reconciliation rather than false success.
DAIL-EXP-AC-008	AI-assisted experiments cannot directly verify, approve, promote, or create trusted state.
DAIL-EXP-AC-009	Results are reproducible from recorded source, artifact, data, graph, policy, and configuration.
DAIL-EXP-AC-010	Failed and aborted experiments remain auditable.
DAIL-EXP-AC-011	Adoption requires normal CI, system testing, security, and deployment controls.
DAIL-EXP-AC-012	Critical evidence-write failures are detected and surfaced.
DAIL-EXP-AC-013	Resource, cost, and execution limits can terminate unsafe/runaway experiments.
DAIL-EXP-AC-014	Production experiments are non-destructive by default and require explicit authorization for exceptions.
DAIL-EXP-AC-015	No experiment path can directly manufacture VERIFIED, APPROVED, PROMOTED, or TRUSTED state.

40. Traceability

Upstream	DAIL Experiment Dependency
00 — Master System	Control loop, trusted-state model, AI non-authority.
01 — SRS	Experimentation, security, reliability, AI, data requirements.
02 — Architecture	DAIL boundary, data flows, workers, isolation, trust boundaries.
03 — Functional	Lifecycle, use cases, async behavior, failure paths.
04 — Product & UX	Impact/verification presentation and operator workflows.
05 — Domain & Data	Experiment, candidate, dependency, evidence, audit objects.
06 — State & Lifecycle	Invariants, invalidation, version binding, reconciliation.
07 — Identity & Dependency	Graph, identity, confidence, impact, freshness.
08 — Security & Threat Model	Threat-driven experiment controls.

09 — Intelligence & AI Architecture	AI abstraction and bounded AI evaluation.
10 — LLM Integration	Provider behavior, routing, failure, cost.
11 — AI Safety	Injection, tool safety, guardrail evaluation.
12–26 — DAIL Package	Core DAIL architecture, verification, evidence, testing, DevOps, implementation.
27 — AWS & Terraform	Controlled AWS infrastructure/sandbox testing.
28 — API	Contract, lifecycle, authorization testing.
29 — Cloud Integration	Provider behavior, mutation ambiguity, reconciliation.
30 — External Integration	External content/connector behavior.
31 — Observability	Experiment telemetry, evidence, audit.
32 — Security Operations	Security experimentation, detection, incident linkage.
33 — Disaster Recovery	Recovery testing and experiment-state recovery.
34 — Deployment & DevOps	Normal delivery of adopted experimental code.
35 — System Testing	System-level validation and gates.

41. Engineering Decisions

- DAIL experimentation is controlled and never a production bypass.
- Control/treatment comparisons use immutable versioned inputs.
- Safety metrics are first-class constraints.
- Experimental output is never authoritative solely because it performs better statistically.
- AI may assist DAIL experiments but cannot become verification or promotion authority.
- Real AWS behavior is tested in dedicated scopes where mocks are insufficient.
- Failed and aborted experiments preserve evidence.
- Adoption requires normal engineering, security, testing, deployment, and governance controls.
- Reproducibility is a quality property for important experimental conclusions.

42. Definition of Done

- Experiment schema and registry are implemented.
- Control/treatment execution is isolated and versioned.
- Ground-truth DAIL datasets/fixtures exist.
- Dependency, impact, verification, lifecycle, evidence, performance, and resilience experiments exist.
- Safety gates and abort conditions are implemented.
- Experiment identities/environments are least-privileged and isolated.
- AI-assisted experiments include injection, grounding, safety, and non-authority tests.
- Evidence is durable, auditable, and reproducible.
- Reports support explicit decisions.
- Adopted code flows through Specification 35 testing and Specification 34 deployment.
- Recovery scenarios are tested where operationally required.
- Traceability to Specifications 00–35 is maintained.

43. Glossary

Term	Definition
Experiment	Controlled investigation comparing treatments against a defined baseline.
Treatment	Experimental implementation or configuration.
Control	Reference implementation used for comparison.
Hypothesis	Predeclared testable prediction.
Ground Truth	Trusted reference result used to evaluate correctness.
Replay	Execution against a fixed historical/synthetic input.

Safety Metric	Metric whose regression can block adoption regardless of average performance.
DAIL	CloudShield AI subsystem for dependency-aware lifecycle, impact, verification and related controls.
Impact Analysis	Determination of resources/relationships affected by a proposed change.
Verification	Deterministic validation of candidate conditions.
Promotion	Governed transition toward applying a verified change.
Trusted State	State earned only after verification, governance, promotion, and authoritative confirmation.
Reconciliation	Resolution of differences between expected and authoritative observed state.
Experiment Evidence	Durable records of execution, inputs, configuration, and results.
Adoption	Decision to move a treatment into the normal engineering/release pipeline.

Final experiment rule: experimentation may discover better behavior, but the authority chain remains unchanged — experiment → evidence → engineering decision → normal testing → governed deployment → authoritative verification. Experimental results never become trusted state by themselves.